Follow step by step to imitate session 1 project (be careful with spelling mistakes in code and line breaks, when copying terminal code)

1. **Open VS Code and then terminal inside VS Code**

2. **Inside terminal enter:**

mkdir healthcare_agent
cd healthcare_agent

3. **Create a virtual environment:**

Enter in terminal:
python -m venv .venv
If u r using python version 3.13, try below version instead of above:
python -m venv .venv --without-pip
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip

Activate it:
.\.venv\Scripts\Activate
You should now see `(.venv)` in front of your terminal prompt.

4. **Create Base folder structure:**

mkdir core,data,models,scripts,storage,ui
ni requirements.txt
ni core__init__.py
ni ui__init__.py

Explanation:

- **`mkdir` creates folders**
- **`ni` means `New-Item`, which creates files in PowerShell**

5. **Add requirements.txt**

Run in terminal:

@"

sentence-transformers

faiss-cpu

pandas

numpy

scikit-learn

gradio

folium

openai

"@ | Set-Content requirements.txt

After running above install packages using:

pip install -r requirements.txt

6. **Create `core/config.py`**

Run below in terminal:

```
@"

import os


BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


DATA_DIR = os.path.join(BASE_DIR, "data")

MODELS_DIR = os.path.join(BASE_DIR, "models")

STORAGE_DIR = os.path.join(BASE_DIR, "storage")


MEDICAL_KB_PATH = os.path.join(DATA_DIR, "medical_knowledge.csv")

DOCTORS_PATH = os.path.join(DATA_DIR, "doctors.csv")
```

```
INSURANCE_RAW_PATH = os.path.join(DATA_DIR, "insurance.csv")


INSURANCE_MODEL_PATH = os.path.join(MODELS_DIR, "insurance_model.pkl")

SYMPTOM_INDEX_PATH = os.path.join(MODELS_DIR, "symptom_index.faiss")

SYMPTOM_EMBEDDINGS_PATH = os.path.join(MODELS_DIR,
"symptom_embeddings.npy")


PATIENT_MEMORY_PATH = os.path.join(STORAGE_DIR, "patient_memory.json")


EMBEDDING_MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"

TOP_K_DIAGNOSES = 5


SAFETY_DISCLAIMER = (

    "This system is for informational purposes only and does not
provide "

    "a medical diagnosis. Always consult a licensed healthcare
professional."

)


SECRETS_PATH = os.path.join(BASE_DIR, "secrets.json")

"@ | Set-Content core\config.py
```

Why this file matters:

This is the central settings file. Every other module will import paths and constants from here.

7. **Create `data/medical_knowledge.csv`**

Run below in terminal:

```
@"

id,symptom_pattern,diagnosis,specialty,triage_level,red_flags,recommen
ded_tests,advice,icd_code

1,"chest pain, shortness of breath, sweating","Possible angina or
heart-related issue","Cardiology","emergency","Severe chest pain,
radiating to arm/jaw, sweating, nausea, or shortness of breath","ECG,
troponin blood test, chest X-ray","Seek emergency care immediately. Do
not ignore or self-medicate.","I20"

2,"fever, cough, sore throat, body ache","Upper respiratory infection
(e.g., flu)","Internal Medicine","non-urgent","Persistent high fever >
3 days, difficulty breathing, confusion, chest pain","Rapid flu test,
CBC, chest X-ray if breathing issues","Hydration, rest, and fever
control. See a doctor if symptoms worsen.","J11"

3,"frequent urination, extreme thirst, unexplained weight
loss","Possible diabetes mellitus","Endocrinology","urgent","Blurred
vision, confusion, fruity breath smell, very high blood
sugar","Fasting blood sugar, HbA1c, urine ketones","Schedule a prompt
appointment with an endocrinologist.","E11"

4,"lower back pain after lifting, no leg weakness or
numbness","Mechanical low back
pain","Orthopedics","non-urgent","Weakness, numbness, loss of bladder
or bowel control","Physical exam, X-ray if persistent, MRI if
neurological signs","Rest, posture correction, and medical review if
pain persists.","M54"

"@ | Set-Content data\medical_knowledge.csv
```

Why this matters:

This is your structured medical knowledge base.
Without this, retrieval has nothing to search.

8. **Create `data/doctors.csv`**

Run in terminal:

```
@"
name,specialty,city,rating,latitude,longitude
Dr. Park Jihoon,Cardiology,Seoul,4.7,37.5665,126.9780
Dr. Kim Hana,Internal Medicine,Seoul,4.6,37.5665,126.9785
Dr. Lee Minsoo,Endocrinology,Seoul,4.5,37.5650,126.9770
Dr. Choi Sunwoo,Orthopedics,Seoul,4.4,37.5670,126.9790
Dr. Nguyen An,Internal Medicine,Hanoi,4.6,21.0278,105.8342
Dr. Tran Minh Duc,Cardiology,Hanoi,4.7,21.0285,105.8350
Dr. Le Thu Ha,Endocrinology,Ho Chi Minh City,4.5,10.7769,106.7009
Dr. Pham Gia Bao,Orthopedics,Da Nang,4.4,16.0544,108.2022
"@ | Set-Content data\doctors.csv
```

Why this matters:

This is the action layer. AI should recommend next steps, not just predictions.

9. **Create `scripts/__init__.py`**

Run in terminal:

```
ni scripts\__init__.py
```

10. **Create `scripts/build_knowledge_index.py`**

Run in terminal:

```
@"
import numpy as np
import pandas as pd
```

```
from sentence_transformers import SentenceTransformer

import faiss

import os


from core.config import (

    MEDICAL_KB_PATH,

    SYMPTOM_INDEX_PATH,

    SYMPTOM_EMBEDDINGS_PATH,

    EMBEDDING_MODEL_NAME,

)


def build_index():

    if not os.path.exists(MEDICAL_KB_PATH):

        raise FileNotFoundError(f"Medical knowledge file not found at
{MEDICAL_KB_PATH}")


    df = pd.read_csv(MEDICAL_KB_PATH)


    if "symptom_pattern" not in df.columns:

        raise ValueError("medical_knowledge.csv must contain a
symptom_pattern column")


    texts = df["symptom_pattern"].astype(str).tolist()
```

```
    print(f"Loaded {len(texts)} symptom patterns")
    model = SentenceTransformer(EMBEDDING_MODEL_NAME)

    print("Encoding symptom patterns...")
    embeddings = model.encode(texts, show_progress_bar=True)
    embeddings = np.array(embeddings).astype("float32")

    dim = embeddings.shape[1]
    index = faiss.IndexFlatL2(dim)
    index.add(embeddings)

    os.makedirs(os.path.dirname(SYMPTOM_INDEX_PATH), exist_ok=True)

    faiss.write_index(index, SYMPTOM_INDEX_PATH)
    np.save(SYMPTOM_EMBEDDINGS_PATH, embeddings)

    print("Knowledge index built successfully.")

if __name__ == "__main__":
    build_index()
"@ | Set-Content scripts\build_knowledge_index.py
```

Why this matters:

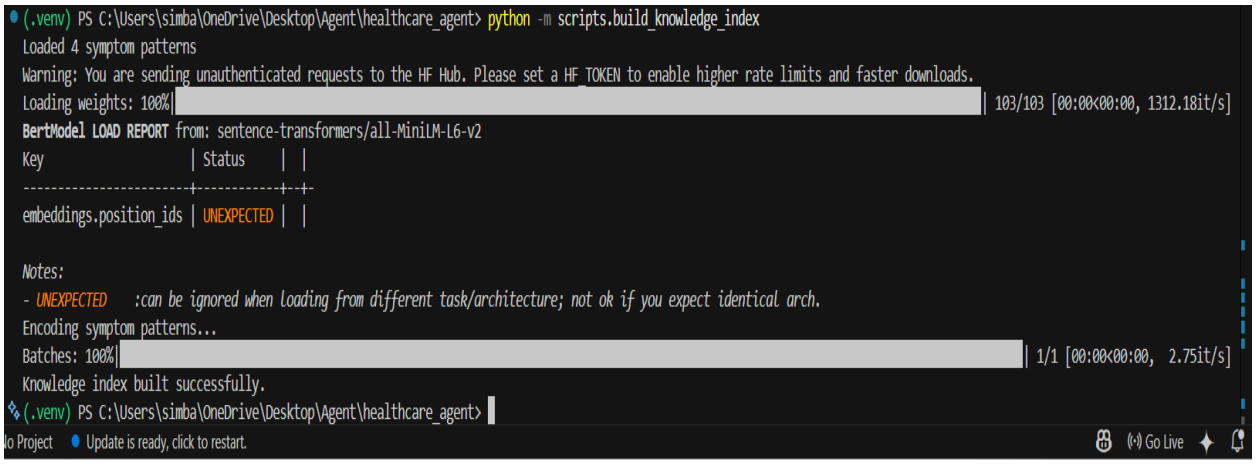
This script converts medical text into vectors and stores a FAISS index for semantic search.

11. Build the knowledge index

Run in terminal:

```
python -m scripts.build_knowledge_index
```

Expected result:



12. **Create `core/memory.py`**

Run in terminal:

```
@"
import json
import os
import hashlib
from datetime import datetime
from typing import Dict, Any, Optional, List

from .config import PATIENT_MEMORY_PATH, STORAGE_DIR

def generate_patient_id(name: str, email: str) -> str:
    key = f"{name.strip().lower()}|{email.strip().lower()}"
    return hashlib.sha256(key.encode("utf-8")).hexdigest()[:16]

def load_memory() -> Dict[str, Any]:
```

```
    os.makedirs(STORAGE_DIR, exist_ok=True)
    if not os.path.exists(PATIENT_MEMORY_PATH):
        return {}
    try:
        with open(PATIENT_MEMORY_PATH, "r", encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError:
        return {}

def save_memory(data: Dict[str, Any]) -> None:
    os.makedirs(STORAGE_DIR, exist_ok=True)
    with open(PATIENT_MEMORY_PATH, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def get_patient_history(patient_id: str) -> Optional[Dict[str, Any]]:
    memory = load_memory()
    return memory.get(patient_id)

def update_patient_memory(
    patient_id: str,
    demographics: Dict[str, Any],
    symptoms: str,
    diagnosis_result: Dict[str, Any],
    insurance_price: Optional[float],
    doctors: List[Dict[str, Any]],
) -> None:
    memory = load_memory()

    if patient_id not in memory:
        memory[patient_id] = {
            "demographics": demographics,
            "history": []
        }

    entry = {
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "symptoms": symptoms,
        "diagnosis": diagnosis_result,
```

```
        "insurance_price": insurance_price,
        "doctors": doctors,
    }

    memory[patient_id]["history"].append(entry)
    save_memory(memory)
"@ | Set-Content core\memory.py
```

Why this matters:

This gives the agent memory, which is one of the strongest differences between an agent and a one-shot predictor.

13. **Create core/diagnostics.py**

```
@"
from typing import Dict, Any, Optional, List

import pandas as pd
import faiss
from sentence_transformers import SentenceTransformer

from .config import (
    MEDICAL_KB_PATH,
    SYMPTOM_INDEX_PATH,
    EMBEDDING_MODEL_NAME,
    TOP_K_DIAGNOSES,
    SAFETY_DISCLAIMER,
)
from .memory import get_patient_history

class SymptomDiagnosticEngine:
    def __init__(self):
        self._df = pd.read_csv(MEDICAL_KB_PATH)
        self._model = SentenceTransformer(EMBEDDING_MODEL_NAME)
        self._index = faiss.read_index(SYMPTOM_INDEX_PATH)

    def diagnose(self, symptoms: str, patient_id: Optional[str] =
None) -> Dict[str, Any]:
        query_vec = self._model.encode([symptoms]).astype("float32")
```

```
        distances, indices = self._index.search(query_vec,
TOP_K_DIAGNOSES)

        distances = distances[0]
        indices = indices[0]

        matches: List[Dict[str, Any]] = []
        for dist, idx in zip(distances, indices):
            if idx < 0 or idx >= len(self._df):
                continue

            row = self._df.iloc[idx]
            confidence = float(1.0 / (1.0 + dist))

            matches.append({
                "id": int(row["id"]),
                "symptom_pattern": str(row["symptom_pattern"]),
                "diagnosis": str(row["diagnosis"]),
                "specialty": str(row["specialty"]),
                "triage_level": str(row["triage_level"]),
                "red_flags": str(row["red_flags"]),
                "recommended_tests": str(row["recommended_tests"]),
                "advice": str(row["advice"]),
                "icd_code": str(row.get("icd_code", "")),
                "distance": float(dist),
                "confidence": confidence,
            })

        matches_sorted = sorted(matches, key=lambda m:
m["confidence"], reverse=True)
        top_diagnosis = matches_sorted[0] if matches_sorted else None

        history_summary = ""
        if patient_id:
            history = get_patient_history(patient_id)
            if history and history.get("history"):
                visits = history["history"]
```

```
                history_summary = f"This patient has {len(visits)}
previous recorded visit(s)."

        return {
            "top_diagnosis": top_diagnosis,
            "matches": matches_sorted,
            "history_summary": history_summary,
            "safety_disclaimer": SAFETY_DISCLAIMER,
        }
"@ | Set-Content core\diagnostics.py
```

Why this matters:

This is the actual reasoning layer.
It retrieves the closest medical condition and returns structured guidance.

14. **Quick diagnosis test**

Run in Terminal:

```
python -c "from core.diagnostics import SymptomDiagnosticEngine;
engine=SymptomDiagnosticEngine(); print(engine.diagnose('chest pain
and sweating'))"
```

If this works, retrieval is alive.

15. **Create core/doctors.py**

Run in Terminal:

```
@"
from typing import List, Dict
import pandas as pd
import folium

from .config import DOCTORS_PATH

def load_doctors() -> pd.DataFrame:
    return pd.read_csv(DOCTORS_PATH)

def find_doctors(specialty: str, city: str, top_k: int = 3) ->
List[Dict]:
```

```
    df = load_doctors()

    filtered = df.copy()

    if specialty:
        filtered =
filtered[filtered["specialty"].str.contains(specialty, case=False,
na=False)]

    if city:
        filtered = filtered[filtered["city"].str.contains(city,
case=False, na=False)]

    if filtered.empty and specialty:
        filtered = df[df["specialty"].str.contains(specialty,
case=False, na=False)]

    if filtered.empty:
        filtered = df[df["specialty"].str.contains("Internal",
case=False, na=False)]

    filtered = filtered.head(top_k)

    return [
        {
            "name": row["name"],
            "specialty": row["specialty"],
            "city": row["city"],
            "rating": float(row["rating"]),
            "latitude": float(row["latitude"]),
            "longitude": float(row["longitude"]),
        }
        for _, row in filtered.iterrows()
    ]

def create_doctor_map(doctors: List[Dict]) -> str:
    if not doctors:
        return "<p>No doctors found.</p>"
```

```
    first = doctors[0]
    m = folium.Map(location=[first["latitude"], first["longitude"]],
zoom_start=12)

    for doc in doctors:
        folium.Marker(
            location=[doc["latitude"], doc["longitude"]],
            popup=f"{doc['name']} ({doc['specialty']}) - {doc['city']}
⭐{doc['rating']}",
        ).add_to(m)

    return m._repr_html_()
"@ | Set-Content core\doctors.py
```

16. **Quick doctor test**

Run in Terminal:

```
python -c "from core.doctors import find_doctors;
print(find_doctors('Cardiology','Seoul'))"
```

17. **Add insurance dataset**

Run in Terminal:

```
ni data\insurance.csv
```

Then let's populate this file:

```
@"
age,sex,bmi,children,smoker,region,charges
19,female,27.9,0,yes,southwest,16884.92
18,male,33.77,1,no,southeast,1725.55
28,male,33.0,3,no,southeast,4449.46
33,male,22.7,0,no,northwest,21984.47
32,male,28.88,0,no,northwest,3866.86
31,female,25.74,0,no,southeast,3756.62
46,female,33.44,1,no,southeast,8240.59
37,female,27.74,3,no,northwest,7281.51
37,male,29.83,2,no,northeast,6406.41
60,female,25.84,0,no,northwest,28923.14
25,male,26.22,0,no,northeast,2721.32
62,female,26.29,0,yes,southeast,27808.73
```

```
23,male,34.4,0,no,southwest,1826.84
56,female,39.82,0,no,southeast,11090.72
27,male,42.13,0,yes,southeast,39611.76
19,male,24.6,1,no,southwest,1837.24
52,female,30.78,1,no,northeast,10797.34
23,male,23.85,0,no,northwest,2395.17
56,male,40.3,0,no,southwest,10602.39
30,male,35.3,0,yes,southwest,36837.47
34,female,31.92,1,no,northeast,3771.21
59,male,27.72,3,no,southeast,14001.13
63,female,23.08,0,no,northeast,14451.84
55,female,32.78,2,no,northwest,12268.63
40,male,28.6,1,no,southwest,5910.94
29,female,30.5,2,no,southeast,5124.19
48,male,37.1,1,yes,southeast,33475.82
36,female,26.7,0,no,northeast,5266.37
41,male,31.1,2,no,northwest,7382.55
50,female,34.2,1,yes,southeast,39836.52
"@ | Set-Content data\insurance.csv
```

This dataset contains the main patterns we should notice:

- **older age tends to increase cost**
- **higher BMI can increase cost**
- **smoker = yes strongly increases cost**
- **sex, region, children are extra structured features**

  "The model is trying to learn relationships between health-related factors and estimated insurance charges."

18. Verify file exists:

```
Get-Content data\insurance.csv | Select-Object -First 5
```

You should see the header and first few rows.

18.1: Create scripts/train_insurance_model.py

Run in terminal:

```
@"

import os

import pickle


import pandas as pd

from sklearn.ensemble import RandomForestRegressor

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder


from core.config import INSURANCE_RAW_PATH, INSURANCE_MODEL_PATH,
MODELS_DIR


def train():

    if not os.path.exists(INSURANCE_RAW_PATH):

        raise FileNotFoundError(f"Insurance CSV not found at
{INSURANCE_RAW_PATH}")


    df = pd.read_csv(INSURANCE_RAW_PATH)


    required_cols = {"age", "sex", "bmi", "children", "smoker",
"region", "charges"}

    missing = required_cols - set(df.columns)

    if missing:
```

```
        raise ValueError(f"Missing required columns in insurance.csv:
{missing}")


    X = df[["age", "sex", "bmi", "children", "smoker",
"region"]].copy()

    y = df["charges"].values


    categorical_cols = ["sex", "smoker", "region"]

    encoders = {}


    for col in categorical_cols:

        le = LabelEncoder()

        X[col] = le.fit_transform(X[col].astype(str))

        encoders[col] = le


    numeric_cols = ["age", "bmi", "children"]


    X_train, X_test, y_train, y_test = train_test_split(

        X, y, test_size=0.2, random_state=42

    )


    model = RandomForestRegressor(

        n_estimators=300,
```

```
        random_state=42,
        n_jobs=-1,
    )
    model.fit(X_train, y_train)

    score = model.score(X_test, y_test)
    print(f"R^2 on test set: {score:.4f}")

    bundle = {
        "model": model,
        "encoders": encoders,
        "categorical_cols": categorical_cols,
        "numeric_cols": numeric_cols,
        "feature_order": ["age", "sex", "bmi", "children", "smoker",
"region"],
    }

    os.makedirs(MODELS_DIR, exist_ok=True)
    with open(INSURANCE_MODEL_PATH, "wb") as f:
        pickle.dump(bundle, f)

    print("Insurance model saved successfully.")
```

```
if __name__ == "__main__":

    train()

"@ | Set-Content scripts\train_insurance_model.py
```

19. **Now we train the model:**

Once the file is there, run in terminal:

```
python -m scripts.train_insurance_model
```

If scripts/train_insurance_model.py has already been created, it should train immediately.

Expected output will look like:

```
(.venv) PS C:\Users\simba\OneDrive\Desktop\Agent\healthcare_agent> python -m scripts.train_insurance_model
R^2 on test set: 0.2992
Insurance model saved successfully.
```

20. **Create core\insurance.py**

Run in terminal:

```
ni core\insurance.py
```

Then paste this in terminal:

```
@"
from typing import Tuple
import numpy as np
import pandas as pd
import pickle
import os

from .config import INSURANCE_MODEL_PATH
from .memory import get_patient_history

class InsuranceEstimator:
    def __init__(self):
        if not os.path.exists(INSURANCE_MODEL_PATH):
            raise FileNotFoundError(
```

```
                f"Insurance model not found at {INSURANCE_MODEL_PATH}.
"
                f"Run python -m scripts.train_insurance_model first."
            )

        with open(INSURANCE_MODEL_PATH, "rb") as f:
            bundle = pickle.load(f)

        self.model = bundle["model"]
        self.encoders = bundle["encoders"]
        self.categorical_cols = bundle["categorical_cols"]
        self.numeric_cols = bundle["numeric_cols"]
        self.feature_order = bundle["feature_order"]

    def _build_feature_row(
        self,
        age: int,
        sex: str,
        bmi: float,
        children: int,
        smoker: str,
        region: str,
    ) -> pd.DataFrame:
        data = {
            "age": [age],
            "sex": [sex],
            "bmi": [bmi],
            "children": [children],
            "smoker": [smoker],
            "region": [region],
        }

        df = pd.DataFrame(data)

        for col in self.categorical_cols:
            le = self.encoders[col]
            df[col] = df[col].apply(
```

```
                lambda v: le.transform([v])[0] if v in le.classes_
else 0
            )

        return df[self.feature_order]

    def estimate(
        self,
        patient_id: str,
        age: int,
        sex: str,
        bmi: float,
        children: int,
        smoker: str,
        region: str,
    ) -> Tuple[float, str]:
        features = self._build_feature_row(age, sex, bmi, children,
smoker, region)
        base_pred = float(self.model.predict(features)[0])

        history = get_patient_history(patient_id)
        history_prices = []
        if history and history.get("history"):
            for entry in history["history"]:
                price = entry.get("insurance_price")
                if price is not None:
                    history_prices.append(float(price))

        if history_prices:
            hist_avg = float(np.mean(history_prices))
            blended = 0.7 * base_pred + 0.3 * hist_avg
            explanation = (
                f"Estimated insurance cost: {blended:,.0f} "
                f"(70% current estimate + 30% historical average)."
            )
            return blended, explanation
        else:
            return (
```

```
                base_pred,
                f"Estimated insurance cost: {base_pred:,.0f} (no prior
history)."
            )
"@ | Set-Content core\insurance.py
```

Why this file matters

This file turns the trained model into a reusable prediction engine.

So the logic becomes:

raw user data -> encoded features -> trained model -> predicted insurance estimate

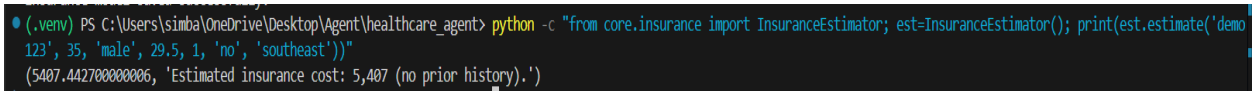
And even better, it introduces a second AI-agent concept:

history-aware prediction

If previous estimates exist, the new value is blended with prior history.

21. **Test the insurance module**

Run in Terminal:

```
python -c "from core.insurance import InsuranceEstimator;
est=InsuranceEstimator(); print(est.estimate('demo123', 35, 'male',
29.5, 1, 'no', 'southeast'))"
```

You should get something like: